Ana DÍaz Jiménez
100522155

Jaime Sánchez Sánchez
10022207

Ingeniería Informática
Grupo:85

PRÁCTICA 1

INTRODUCCIÓN AL LENGUAJE ENSAMBLADOR

Ejercicio 1 – Compute_integral 3
Funciones relevantes del ejercicio 1 3
FUNCIÓNF 3
COMPUTE_INTEGRAL 4
Pseudocódigo del ejercicio 1 5
Batería de pruebas 6
EJERCICIO 2 – INTEGRAL_MATRIX 8
Funciones relevantes del ejercicio 2 8
INTEGRAL MATRIX 8
Bucle1 8
escero 9
Pseudocódigo del ejercicio 2 10
Batería de pruebas 11

Ejercicio 1 – Compute_integral

El ejercicio 1 consistía en implementar un código en ensamblador el cual almacenara en un registro la solución de la integral de la función provista entre los valores “a” y “b”.
Para ello se debía resolver la función primero, a través de otra función que se llama en la función general Compute_integral. Esta función a su vez llamaba dentro de ella a otra función,” pow”, que requería la entrada de dos valores, “a” y “b” (no los mismos de los que depende la integral) para calcular ciertas partes de la función.

Funciones relevantes del ejercicio 1

FUNCIÓNF

Función principal

Lo primero que hicimos fue comenzar afrontando el funcionamiento de la función “funcionf”, la cual debía hacer lo siguiente:

$$f(x) = px^2 + qx + r$$

En este caso primero hemos sumado “q” multiplicado con “x”, puesto que deducimos que para sumar “p” multiplicado por “x” al cuadrado debíamos emplear la función de “pow” para calcular “x^2”.
Posteriormente observamos que al obtenerse la suma primero debíamos pasar todos los valores a coma flotante para así poder sumarlos entre ellos (puesto que x = a + n*h, y h es un número decimal).
Llegamos a la conclusión de que el resultado final a obtener debía estar en coma flotante así que pasamos todos los valores a coma flotante para su posterior suma, por tanto, el resultado a devolver estaría almacenado en el registro fa0.

Pow(a,b)

Esta función nos hizo darnos cuenta de la necesidad de almacenar todos los datos empleados en una función en la pila antes de llamar a otra dentro de ella, pues nos descolocó todo.
En este caso, primero almacenamos todos los datos de "funciónf" en la pila, incluyendo el "ra" para retornar a nuestra antigua posición posteriormente, luego almacenamos "x" en fa0 y el valor 2 en el registro a0, para así poder calcular "x^2". Una vez devuelto el resultado la función, multiplicamos dicho flotante por "p" (pasando todo previamente a flotante con "fcvt.s.w") y sumamos el resto de los valores de la función en fa0.

COMPUTE_INTEGRAL

Función principal

La función compute integral tiene como objetivo devolver el valor de la integral de la función con los valores obtenidos entre "b" y "a". También se puede expresar como el sumatorio de la función con x = a + h*n, todo ello multiplicado por h.
Se nos dice también que h es igual al valor de b – a todo ello dividido entre N (en este caso N vale 1000).

Deducimos que, dado que h no iba a ser un valor entero y tendríamos que operar con él, debíamos pasar todos los datos a coma flotante para calcular su valor.

Bucle

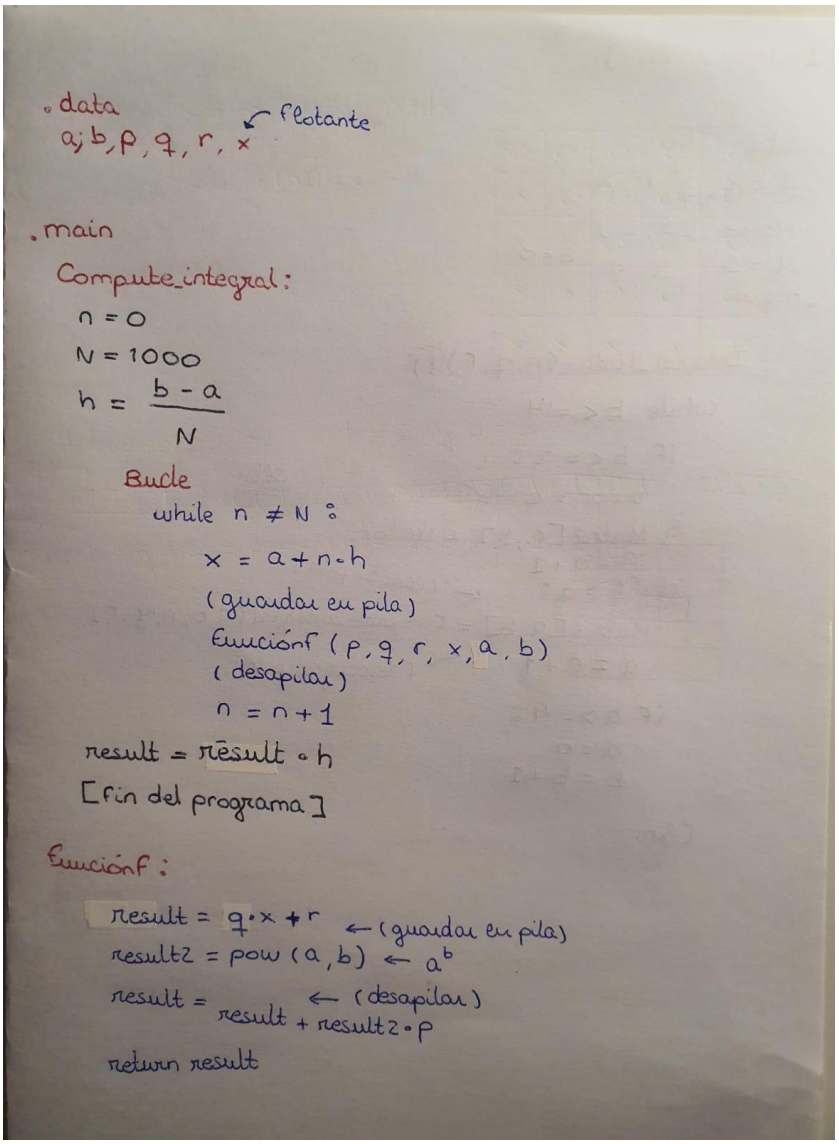
También dentro de Compute_integral implementamos un bucle para calcular la suma de todas las funciones desde que n vale cero hasta que n = N.
Primero calculamos el valor de la "x" (x = a + n*h) que irá variando en función del valor de la n, luego apilamos todos los valores en la pila que vallamos a necesitar posteriormente (junto con ra) y llamamos a la función "funcionf", que nos devuelve el valor de la función en relación con lo que vale "x". Desapilamos y sumamos n + 1, completando una iteración de las 1000 que nos piden hacer. Posteriormente regresamos al bucle hasta que se cumpla la condición final n = N.

Fin

Cuando n se iguale a N, el bucle reconducirá hacia la etiqueta “fin”, que calculará el sumatorio multiplicado por h.

En “fin” tenemos dos posibles opciones:
Cargamos el valor 1 en el registro t3, y luego igualamos este al registro s1.
En caso de que no valgan igual, finalizaremos Compute_integral con a7 = 10 y acabará el programa.
En el caso contrario, se nos redirigirá a la etiqueta “retorno”, empleada en la segunda parte del proyecto.

Pseudocódigo del ejercicio 1



Pseudocódigo empleado para describir el primer ejercicio

Batería de pruebas

Datos introducidos	**Descripción de la prueba**	**Resultado esperado**	**Resultado obtenido**
Q = 5 P = 2	Prueba de los resultados que se	La integral da 70,5	El valor obtenido en fa0 es 30

R = 10 A = 0 B = 3 N = 1	obtienen con la integral de 0 a 3 con 1 repetición para los valores de q, p y r (N es más pequeña para poder controlar mejor el valor)	El sumatorio da 30	
Mismos valores, solo que ahora N vale 100	Probar que cuanto mayor es N, más ajustado al valor de la integral es el valor del sumatorio	La integral da 70, 5 de nuevo El sumatorio muestra un valor de 69,7 (aprox. Se desestimaron decimales pasados las centésimas)	El valor obtenido es de 70,00588 (el ordenador no desestima decimales)
Mismos valores, con N = 1000	Terminar el ejercicio calculando el sumatorio lo más exacto posible con 1000 iteraciones	La integral da 70, 5 de nuevo El sumatorio no se pudo calcular debido a la alta cifra de repeticiones	El valor devuelto por el ordenador es de 70,45045. Muy aproximado al valor de la integral.
Datos cualesquiera, con N = 0	Se hace la operación con un sumatorio con N = 0	Error o resultado 0	No carga nada en ningun valor y suelta “Division by zero not allowed”
Datos cualesquiera, con un jal ra en vez de un jr ra	Se sustituye un jr ra por un jal ra	Fallo de lectura	No identifica Integral_matrix
Datos cualesquiera con a = 65 y b = 3	La integral de un número superior sobre uno inferior debería dar negativo	Devuelve valor negativo	Devuelve valor negativo
Datos cualesquiera con a = 0 y b = 340000000000000000000000000000000000000	Sumatorio muy alto para observar hasta cuando puede llegar el programa. El valor máximo que se puede obtener es 3.4028235×10^38, que es el valor máximo que se puede representar con IEEE 754	Posible valor que no pueda representar la máquina	Devuelve ” Error description: Please check instruction syntax, inmediate ranges, register name, etc.” Suelta un error, probablemente por lo grande que es el valor.

EJERCICIO 2 – INTEGRAL_MATRIX

La segunda parte del proyecto consiste en hacer una matriz de 5x5 y almacenar datos en ella.
Los datos no van a ser otros que los resultados de los sumatorios para N = 100 y un p, q y r por determinar.
La matriz almacenará el valor del sumatorio según en que posición de la matriz nos encontremos: si el número de columna es mayor o igual al número de fila, entonces se almacenará en esa posición un cero. Mientras que, si ocurre al revés, el valor almacenado será el de el resultado de la matriz con b = número superior de la integral y a = número inferior de la integral.
En este caso para calcular el sumatorio en función de a, b, p, q y r, se empleará la función Compute_integral del ejercicio 1.

Funciones relevantes del ejercicio 2

INTEGRAL MATRIX

Lo primero que hacemos en esta función es definir los valores a y b, que serán las filas y columnas, respectivamente, que vayamos avanzando.
Posteriormente cargamos el valor M en un registro, esta muestra cuales van a ser las dimensiones de la matriz.
También cargamos los valores de p, q y r, para almacenarlos como datos de importancia que requiere la función.

Bucle1

Este forma parte de la función y su fin es avanzar filas y columnas para ir rellenando la matriz de valores.
Primero comparamos el número de filas que tenemos con el tamaño de la matriz. Si este número iguala o excede el tamaño de M entonces redirige a “fin1”, mientras que si no lo hace procederemos a catalogar los valores para saber si insertamos un cero o el valor del sumatorio en nuestra matriz.

En caso de que se cumpla b<= a, se direccionará a la etiqueta “escero”, mientras que si se cumple el caso contrario y b>a, se direccionará PC a la etiqueta “integral”

escero

Esta etiqueta calcula la posición en la que se almacenará nuestro valor. Para saber en qué posición debemos almacenar, puesto que el ordenador no identifica lo que es una matriz y lo cataloga como una “lista”, multiplicamos el número de filas que llevamos avanzadas por la longitud máxima de estas y le sumamos el número de columna en el que se halla el valor. Por ejemplo, si la posición es [0,4], calculamos 0*M + 4 y almacenamos el número en dicha posición, con “sw t2, 0 (t1)”, teniendo t2 el valor cero (como se estipula en el ejercicio cuando b<= a) y siendo t1 la matriz más el número de filas multiplicado por el ancho más la columna.ç
Se regresa al bucle al final y se le suma a “a” +1.
Importante realizar un “slli t0, t0, 2” (t0 = b*M +a) para alinear, si no dará error.

integral

Al igual que en “escero”, “integral” tiene la misma ejecución para almacenar datos, calculando b*M +a y poniendo el valor en dicha posición de la matriz. No obstante, en este caso, se ha de calcular Computer_integral con los valores de a,b,p,q y r.

Por tanto, primero almacenamos todos los datos en la pila para poder seguir utilizándolos después de ejecutar la función.

Posteriormente con el ejercicio 1, calculamos el valor del sumatorio y lo devolvemos en fa0, con la etiqueta de retorno anteriormente mencionada.

Volvemos a des apilar todos los datos de interés y con ellos calculamos la posición que tendía el valor en la matriz y lo insertamos.

Sumamos “a” +1 y regresamos al inicio del bucle.

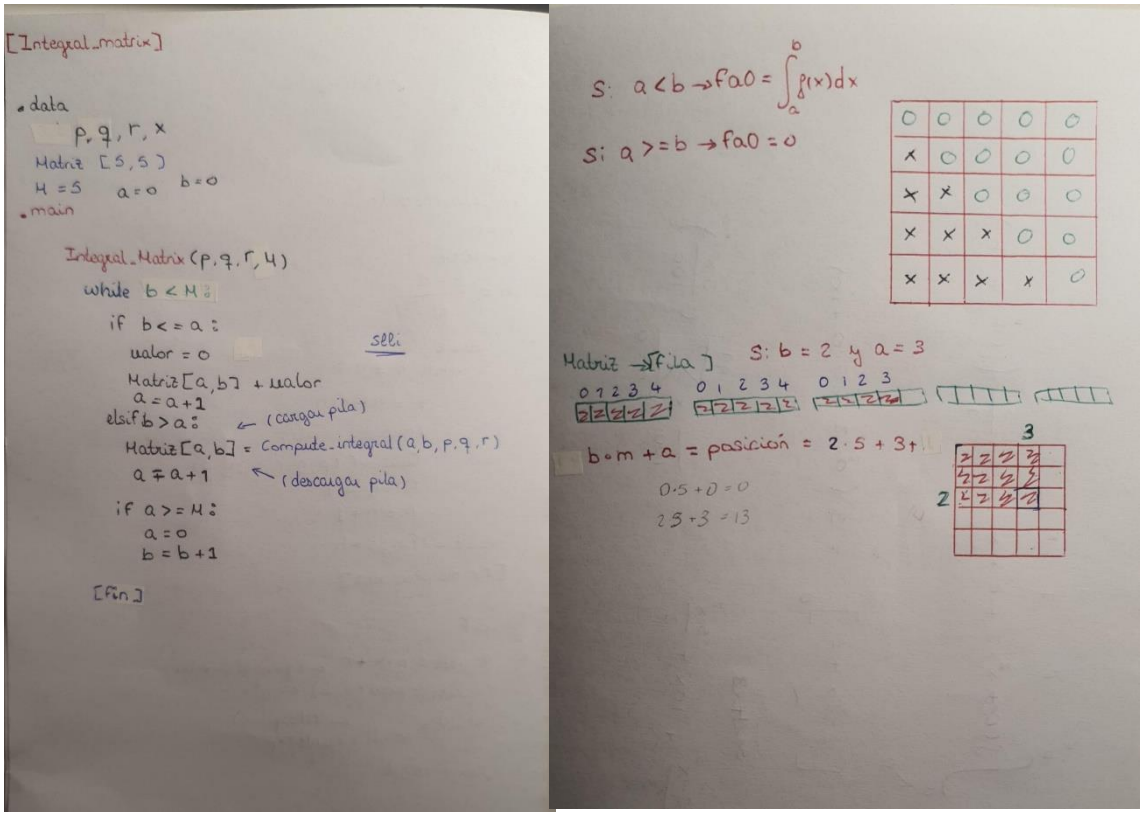
Fin1 y Fin2

En caso de que al inicio del bucle a sea igual o mayor que M (ancho y alto de la matriz), se redirecciona a la etiqueta “fin1”, en la cual se iguala el valor de “a” a cero y se le suma a “b” +1.

En caso de que b sea menor a M, se regresa al bucle y se reinicia el circuito, hasta que b sea igual o mayor a M. En tal caso, se ha recorrido toda la función, y por tanto cada espacio de la matriz tiene un valor.

Como ya se han rellenado todos los huecos, se salta a “fin2” donde se carga el valor 10 al registro “a7” y se hace un “ecall”, poniendo fin al programa.

Pseudocódigo del ejercicio 2



Pseudocódigo del ejercicio 2, junto con explicación gráfica de cómo se debe rellenar la matriz y que datos tiene según número de columna y fila.

Batería de pruebas

Datos introducidos	Descripción de la prueba	Resultado esperado	Resultado obtenido
Datos aleatorios con valor N = -1	Hacer un sumatorio con un total de –1	Error o no almacenaje de datos	Cuando b > a, fa0 se vuelve cero y los almacena en la matriz (matriz vacía)
Datos aleatorios con M = 2	La matriz a rellenar es una 2x2	Almacenará solo un dato distinto de cero	Sólo la posición [0,1] de la matriz tiene un valor diferente de cero
Datos aleatorios, siendo "a" mayor que “b”, pero siendo a menor que “M”, en todo momento	Al cumplirse dicha condición la matriz se llene de ceros	Matriz sin ceros	Se almacenan 0’s en toda la matriz
Matriz en la que “a” siempre sea menor que “b”, pero “a” y “b” siempre sean menor que “M”.	Al cumplirse dicha condición la matriz se llene de elementos no nulos	Matriz llena de fa0 distintos de 0	Matriz rellena de datos aleatorios distintos de 0